Informe técnico — Pedro Infnet

Fecha: 2026-11-08; actualizaciones T15/T24: 2026-11-09. Proyecto: reconstrucción independiente de asistente de compras sobre piezas ficticias. Estado: núcleo local ejecutado; índice autorizado y evaluaciones remotas pendientes.

Objetivo y datos

Pedro consulta y compara ofertas sintéticas de repuestos de electrodomésticos. El generador usa semilla 20261107 y produce 2640 piezas, 7920 ofertas, 7920 compatibilidades y 47520 precios históricos. Cada escenario copia la base a un SQLite propio con UUID y manifiesto SHA-256. La ficha técnica identifica la fuente sintética; precios y stock vigentes se leen siempre del escenario. No se hacen compras ni pagos.

Arquitectura

La interfaz conserva user_id, session_id y dataset_id. Un borrador estructurado por sesión reúne pieza, cantidad y restricciones explícitas entre turnos. La petición actual prevalece: un cambio de cantidad conserva los límites anteriores y «sin presupuesto/plazo» los elimina. Solo se pregunta por pieza o cantidad cuando faltan. El flujo del chat carga los 12 mensajes previos y recuerdos por usuario, crea un agente OpenAI tools con seis herramientas, analiza una RespuestaCompra Pydantic, verifica todas las ofertas activas y sus evidencias, aplica NLI local a las afirmaciones publicables y solo entonces renderiza. Se permite una corrección interna total. RunnableWithMessageHistory escribe la petición y la respuesta pública; las propuestas rechazadas solo viven en la traza de ejecución. Sin clave, el chat compara mediante el mismo cálculo y verificador exacto.

La comparación local usa el mismo cálculo y el mismo verificador exacto. Presupuesto, precio unitario, flete y total son enteros en centavos. La regla exige stock suficiente y, cuando se indican, plazo no superior al máximo y total no superior al presupuesto; desempata por total, plazo e ID de proveedor. La vista de catálogo presenta 20 piezas por página.

Text-to-query valida un PlanConsulta y compila exclusivamente plantillas SELECT con parámetros; SQLite se abre en modo solo lectura. Para T15 se construye en memoria un índice FAISS local de tres políticas ficticias, con fragmentos, fuentes y filtros de metadatos; el chat lo usa si no hay índice autorizado. T24 reproduce en ese corpus una consulta cuyo fragmento correcto queda fuera del top 4 y una reformulación que lo sitúa primero, con fuente, ID y distancia L2 registrados en docs/t24_recuperacion.md. Las respuestas de ese corpus se etiquetan como ficticias y pasan por verificación de fuente y NLI. El índice preexistente de políticas se carga solo tras revisar manifiesto, hashes, dimensión, métrica, recuento, modelo y confianza explícita para el docstore pickle. Ese índice autorizado aún no está disponible.

Anatomía, tipo y madurez del agente (R01)

Anatomía. El ambiente es un escenario aislado: una copia SQLite del catálogo con UUID y manifiesto, el corpus de políticas y el servicio ViaCEP. El motor cognitivo es ChatOpenAI (gpt-4.1-mini por defecto). El system message sigue PTCF e incluye ejemplos few-shot. Cuatro herramientas actúan como sensores (buscar_productos, consultar_catalogo, consultar_politicas y consultar_cep), una calcula (comparar_ofertas) y una escribe (guardar_preferencia). La memoria combina una ventana de 12 mensajes y un borrador por sesión con preferencias y episodios vectoriales por usuario. El orquestador es LangChain: create_openai_tools_agent, AgentExecutor con seis iteraciones como máximo y RunnableWithMessageHistory. Una puerta de publicación aplica verificación exacta y NLI local. Esta anatomía se lee de los objetos reales del agente en src/pedro/anatomia.py y su salida está guardada en el notebook 01.

Tipo. Pedro es un agente híbrido. La capa reactiva son reglas fijas: pedir la cantidad que falta, listar el catálogo, comparar ofertas en centavos y enrutar las preguntas de entrega parcial. La capa deliberativa es el LLM, que elige herramientas, mantiene estado entre turnos y corrige una vez su propuesta frente a la verificación. No es reactivo puro porque conserva estado y elige herramientas; no es deliberativo puro porque no construye un plan explícito de varios pasos. El notebook 01 lo demuestra con un modelo doble guionado: dos turnos se resuelven por regla con cero llamadas al modelo y un tercero pasa por el modelo, que recibe seis herramientas y ejecuta consultar_cep.

Madurez. Se usa una escala genérica de cinco niveles (chain fija, enrutamiento, agente con herramientas, agente con memoria y autoverificación, sistema multiagente autónomo) como supuesto declarado, porque la escala de la primera clase no está en este workspace. La madurez actual es el nivel 4 en diseño, demostrado solo en local. La madurez objetivo es el mismo nivel 4 demostrado con mediciones: trayectorias y calidad en LangSmith, ninguna recomendación publicada sin pasar la verificación y una sesión multi-turno con el modelo real. No se busca el nivel 5: el problema no requiere varios agentes ni ejecutar compras. El detalle, con la arquitectura según CoALA, está en docs/respuestas_rubricas.md.

Planificación y memoria aumentada (R02)

Planificación. Una solicitud se divide en cinco subtareas con salida observable: extraer las restricciones del mensaje, consultar los datos, comparar las ofertas, verificar la propuesta y publicar o corregir. Una compra con ID y cantidad sigue ese plan fijo sin modelo. Una solicitud que mezcla acciones la planifica el LLM dentro del AgentExecutor, con seis iteraciones como máximo y un plan visible en el prompt. No se añadió un planificador explícito porque las subtareas del dominio son conocidas y el plan fijo es más verificable.

Memoria. El corto plazo vive en la sesión: el mensaje actual, el borrador de compra y la ventana de 12 mensajes. El largo plazo vive en el usuario: preferencias explícitas, episodios vectoriales de recomendaciones verificadas y, como memoria procedimental, el prompt y las reglas. Precios, stock y plazos no son memoria: siempre se releen del escenario activo. El notebook 01 guarda la ejecución: al pedir «ahora 3 unidades» el borrador conserva el plazo de 7 días, el historial tiene 18 mensajes y el agente ve 12, y una sesión nueva empieza sin borrador pero con la preferencia del usuario. Con el modelo real (gpt-4.1-mini), una sesión por turnos planificó bien la compra, el CEP y el cambio de cantidad; la pregunta general de política no se publicó porque el modelo citó el fragmento casi literal y la verificación exigía texto idéntico. Se corrigió con citas ancladas: el modelo elige la fuente y el sistema sustituye la cita por el tramo literal del fragmento recuperado; si el resumen no obtiene entailment, se publica solo la cita textual. Repetida la pregunta, la respuesta se publicó con dos fuentes (docs/citas_ancladas.md). Una solicitud mixta en un solo turno sigue sin publicarse y queda declarada como limitación.

Salida estructurada y laboratorio de salidas rotas (R04)

La cadena de extracción convierte una solicitud en un objeto Extraccion validado con Pydantic. Para comprobar su robustez se construyó un laboratorio con ocho salidas típicas de un modelo. Cada defecto lo resuelve una capa distinta: el parser tolera cercas de markdown; una segunda cadena, montada con with_fallbacks, repara los errores de sintaxis y de esquema a partir de la solicitud, la salida inválida y el error; y una regla de dominio recalcula los faltantes sin confiar en el modelo. Con gpt-4.1-mini como reparador, las cinco reparaciones devolvieron los datos correctos. El laboratorio reveló que el parser acepta JSON truncado y pierde datos sin avisar: una salida cortada dejó la solicitud sin su plazo de 7 días. La cadena robusta añade una comprobación estricta y envía ese caso a reparación. También se ejecutó with_structured_output sobre las mismas solicitudes, con resultado idéntico; se mantiene el parser porque permite ver y reparar la salida cruda y probar sin credenciales.

Interfaz Runnable y composición imperativa (R05)

Los tres métodos de Runnable se ejecutaron con gpt-4.1-mini. Con invoke, una solicitud produjo un objeto validado. Con batch, tres solicitudes tardaron 1,43 s frente a 4,55 s de tres invoke en serie, con los mismos resultados. Con stream, una cadena con parser de JSON parcial entregó 19 estados, el primero a los 0,45 s de 1,07 s; la cadena con parser Pydantic entregó un único fragmento, porque necesita el JSON completo para validarlo. Son mediciones únicas, no un benchmark. La composición imperativa usa @chain en dos cadenas: una simple y otra que repara errores con try/except. Esta última resolvió los ocho casos del laboratorio igual que la versión declarativa.

Composición declarativa y elección del patrón (R06)

Hay cuatro cadenas LCEL: una secuencia simple, una con reparación mediante with_fallbacks, una que transmite JSON parcial y una con ramas paralelas. La cadena con reparación existe además en versión imperativa con @chain. Las dos dan el mismo resultado en los ocho casos del laboratorio; lo que cambia es lo observable. En la versión declarativa la traza registra 16 pasos cuando hay reparación, incluido el error del parser y la validación de dominio, y el grafo muestra el fallback. En la imperativa registra 9 y no ve ni el error ni la validación, porque el try/except y la regla son Python normal. Por eso LCEL es el patrón principal para las tuberías. El código imperativo se reserva para orquestar el turno en crear_flujo, que tiene salidas anticipadas y un bucle de dos propuestas, y donde lo que LangChain no ve, como el anclaje de citas, se anota a mano en la traza.

Fase 2: agente con herramientas, ReAct y confiabilidad (R07–R12)

Orquestadores (R07). Se comparan LangChain, LangGraph, LlamaIndex, AutoGen, Semantic Kernel y Langflow por autonomía, abstracción y modularidad. LangChain se mantiene porque Pedro es un solo agente que necesita muchas piezas distintas bajo control estricto: el proyecto usa 13 tipos de componentes suyos, según un inventario leído de los imports. La comparación es conceptual; solo LangChain se ejecutó. Tres de las APIs que pide la rúbrica están marcadas como obsoletas y la migración natural sería a LangGraph.

ReAct (R08). Un agente construido con initialize_agent y ZERO_SHOT_REACT_DESCRIPTION, con dos herramientas y ConversationBufferMemory, se ejecutó con gpt-4.1-mini. Con el historial dentro del prompt resolvió una pregunta de seguimiento (CentroPeças, 2 días); con el prompt zero-shot original, que no lee la memoria, pidió de nuevo el identificador de la pieza.

Routing (R09). Dos routers sobre RouterChain, uno por reglas y otro decidido por el modelo, se compararon en siete preguntas: reglas 2 de 7, LLM 6 de 7. Las preguntas se eligieron para poner a prueba las reglas.

API externa (R10). El agente llama a ViaCEP real por function calling. Se guardaron el esquema enviado al modelo, los argumentos, la respuesta de la API y la respuesta publicada en tres casos: válido, inexistente e inválido. La primera ejecución no publicó la ciudad porque el modelo copiaba mal un identificador; la respuesta se rellena ahora con lo que devolvió la herramienta.

Prompts (R11). El prompt del agente sigue PTCF con cuatro ejemplos. En diez solicitudes difíciles, la extracción acertó 6 sin ejemplos, 10 con few-shot y 10 con few-shot y chain-of-thought; el razonamiento queda escrito antes de los datos.

Confiabilidad (R12). Parsers con reparación, memoria que guarda solo lo publicado, búsqueda FAISS sobre el índice guardado y un callback que registra cada herramienta y cada verificación. Ese registro permitió encontrar y corregir dos fallos reales. El detalle de cada rúbrica está en docs/respuestas_rubricas.md.

Enseñanza y comparación

Los notebooks 01–05 cubren extracción LCEL y @chain; ReAct clásico y RouterChain; memoria CoALA; SQL con una demostración FAISS sintética para T15; y el integrador. El ReAct textual, ConversationBufferMemory y FakeListLLM son demostraciones didácticas. El integrador usa herramientas ligadas al escenario y una puerta de publicación. AutoGen, Mem0 y Letta se comparan conceptualmente en docs/comparaciones.md; no se atribuyen pruebas ejecutadas a esas bibliotecas. Los ejemplos few-shot del prompt describen acciones observables y no incluyen casos de evaluación.

Verificación observada

La sincronización bloqueada de dependencias terminó correctamente. Sesenta y seis pruebas pasaron y los cinco notebooks se ejecutaron con NotebookClient; el notebook 04 incluye T15 y T24. Streamlit respondió ready por Agent API en Chat, Catálogo y Comparación local; la comparación de dos unidades de PEC-01-0001 con R$ 100000 y 60 días eligió BR-06 por R$ 2.963,07. En otra sesión del Agent API, «Compara PEC-01-0001» pidió solo cantidad; «2» eligió BR-06 por R$ 2.963,07 y «ahora 3 unidades» recalculó R$ 4.401,99 sin repetir la pregunta. Una llamada HTTP real a ViaCEP para 01001000 devolvió São Paulo/SP. FastEmbed produjo vectores de 384 dimensiones, recuperó un episodio actualizado entre sesiones e indexó 75 fragmentos ficticios para T15. Cinco consultas del notebook cubren reglas generales y ejemplos cotejados con la base de semilla `20261107`; las políticas no representan pedidos ni entregas almacenados.

El experimento T24 dejó `FIC-ENT-01-S05-C01` fuera del top 4 inicial y lo recuperó primero tras reformular, con el mismo índice y top-k. El comando `/t24` mostró este contraste sin clave OpenAI. Antes del ajuste, la pregunta coloquial «¿Se puede mandar en varias cajas?» recibió una aclaración de pieza; una prueba posterior con ChatOpenAI citó la política, pero repetir la pregunta en una sesión existente produjo dos fallbacks genéricos. La versión actual enruta esa intención de entrega de forma determinista a `consultar_politicas`, extrae la regla del fragmento recuperado y la publica solo tras verificar fuente y NLI. En el flujo local, tres repeticiones con borrador de compra pendiente pasaron con NLI real; en Streamlit Agent API pasaron otras tres repeticiones en una sesión nueva. Esa ruta no llamó al LLM remoto. El modelo NLI fijado clasificó tres pares ilustrativos como entailment, contradiction y neutral; una recomendación local completa obtuvo verificación exacta sin errores y NLI positivo. Las pruebas de reparación usan un executor guionado y se registran separadas de las llamadas remotas puntuales; estas no miden la calidad general del LLM.

Riesgos y trabajo dependiente

La guía integrada de fases, temas y rúbricas (guide/) permitió remapear la matriz a las 29 rúbricas R01–R29; el enunciado original en portugués y el índice FAISS preexistente autorizado siguen fuera de este workspace. Por ello la evaluación LangSmith remota no tiene tasas y el fallo de recuperación sobre el corpus exigido no está acreditado. Se prepararon 30 casos derivados sintéticos con referencias congeladas para LangSmith, sin ejecutar evaluación remota. Las demostraciones locales T15/T24 no sustituyen ese índice. La búsqueda semántica opcional de piezas no se implementó; la búsqueda actual es SQL. El modelo NLI se ejecuta en CPU y su descarga inicial puede tardar. Antes de entregar, cotejar la rúbrica de 29 criterios, incorporar el índice autorizado y repetir allí el diagnóstico T24, ejecutar chat y LangSmith reales con referencias y denominadores, grabar el vídeo con webcam y verificar el enlace Drive.